Ma. Angela Gavino BSINT-2 Section D

JAVA OOP LABORATORY EXERCISE CAMPUS CAFE ORDERING SYSTEM

MenuItem.java

```
public abstract class MenuItem {
    protected String name;
    protected double basePrice;
    private String itemCode;

    //Default constructor.

    public MenuItem() {
        this.name = "Unnamed Item";
        this.basePrice = 0.0;
        this.itemCode = "N/A";
    }

   // Parameters
    public MenuItem(String name, double basePrice, String itemCode) {
        this();
        setName(name);
        setBasePrice(basePrice);
        setItemCode(itemCode);
    }

    // Getters
    public abstract double computePrice();
    public String getDescription() {
        return name + " [" + itemCode + "]";
    }

    public String getName() {
        return name;
    }

    // Setters
    public void setName(String name) {
        if (name != null && !name.isBlank()) {
            this.name = name;
        }
```

```
    }


    public double getBasePrice() {
        return basePrice;
    }

    public void setBasePrice(double basePrice) {
        if (basePrice >= 0) {
            this.basePrice = basePrice;
        }
    }

    public String getItemCode() {
        return itemCode;
    }

    public void setItemCode(String itemCode) {
        if (itemCode != null && !itemCode.isBlank()) {
            this.itemCode = itemCode;
        }
    }
}
```

Beverage.java

```
public class Beverage extends MenuItem {
    private String size;

    // Default constructor

    public Beverage() {
        super();
        size = "S";
    }

    // Parameters
    public Beverage(String name, double basePrice,
                    String itemCode, String size) {
        super(name, basePrice, itemCode);
        this.size = "S";
```

```
        setSize(size);
    }

    @Override
    public double computePrice() {
        if (size.equals("L")) {
            return basePrice + 25;
        } else if (size.equals("M")) {
            return basePrice + 15;
        }
        return basePrice;
    }

    @Override
    public String getDescription() {
        return super.getDescription() + " Size: " + size;
    }

    public String getSize() {
        return size;
    }

    // Setters
    public void setSize(String size) {
        if (size != null &&
                (size.equals("S") ||
                 size.equals("M") ||
                 size.equals("L"))) {
            this.size = size;
        }
    }
}
```

Pastry.java

```
public class Pastry extends MenuItem {

    private boolean warmed;

    // Default constructor
```

```
    public Pastry() {
        super();
        warmed = false;
    }

    public Pastry(String name, double basePrice,
                  String itemCode, boolean warmed) {
        super(name, basePrice, itemCode);
        this.warmed = warmed;
    }

    @Override
    public double computePrice() {
        double price = basePrice;
        if (warmed) {
            price += 10;
        }
        return price;
    }

    @Override
    public String getDescription() {
        if (warmed) {
            return super.getDescription() + " (Warmed)";
        }
        return super.getDescription();
    }

    public boolean isWarmed() {
        return warmed;
    }
    public void setWarmed(boolean warmed) {
        this.warmed = warmed;
    }
}
```

Meal.java

```
public class Meal extends MenuItem {

    private boolean upsized;
```

```
private int extraSides;

// Default constructor.

public Meal() {
    super();
    upsized = false;
    extraSides = 0;
}

public Meal(String name, double basePrice,
            String itemCode, boolean upsized, int extraSides) {
    super(name, basePrice, itemCode);
    this.upsized = upsized;
    this.extraSides = 0;
    setExtraSides(extraSides);
}

@Override
public double computePrice() {
    double price = basePrice;
    if (upsized) {
        price += 20;
    }
    price += 12 * extraSides;
    return price;
}

@Override
public String getDescription() {
    String status = upsized ? "Upsized" : "Regular";
    return super.getDescription() + " " +
            status + ", " + extraSides + " side(s)";
}

public boolean isUpsized() {
    return upsized;
}

public void setUpsized(boolean upsized) {
    this.upsized = upsized;
}

public int getExtraSides() {
    return extraSides;
```

```
    }

    public void setExtraSides(int extraSides) {
        if (extraSides >= 0) {
            this.extraSides = extraSides;
        }
    }
}
```

Customer.java

```
public class Customer {

    private String name;
    private boolean student;
    private int loyaltyPoints;

    // Default constructor
    public Customer() {
        this("Guest", false);
    }

    public Customer(String name) {
        this(name, false);
    }

    public Customer(String name, boolean student) {
        this.name = "Guest";
        this.student = student;
        this.loyaltyPoints = 0;
        setName(name);
    }

    public void addPoints(int points) {
        if (points >= 0) {
            loyaltyPoints += points;
        }
    }

    public void addPoints(double amountSpent) {
        if (amountSpent >= 0) {
```

```
            int points = (int) (amountSpent / 50);
            loyaltyPoints += points;
        }
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name != null && !name.isBlank()) {
            this.name = name;
        }
    }

    public boolean isStudent() {
        return student;
    }

    public void setStudent(boolean student) {
        this.student = student;
    }

    public int getLoyaltyPoints() {
        return loyaltyPoints;
    }

    public void setLoyaltyPoints(int loyaltyPoints) {
        if (loyaltyPoints >= 0) {
            this.loyaltyPoints = loyaltyPoints;
        }
    }
}
```

Order.java

```
public class Order {

    private Customer customer;
    private MenuItem item1;
    private MenuItem item2;
```

```
    private MenuItem item3;
    private int itemCount;
    private String orderId;


    public Order(String orderId, Customer customer) {
        if (orderId == null || orderId.isBlank()) {
            throw new IllegalArgumentException("Order ID cannot be
blank.");
        }
        if (customer == null) {
            throw new IllegalArgumentException("Customer cannot be
null.");
        }
        this.orderId = orderId;
        this.customer = customer;
        this.item1 = null;
        this.item2 = null;
        this.item3 = null;
        this.itemCount = 0;
    }

    public boolean addItem(MenuItem item) {
        if (item == null) {
            return false;
        } else if (itemCount >= 3) {
            return false;
        } else if (item1 == null) {
            item1 = item;
        } else if (item2 == null) {
            item2 = item;
        } else {
            item3 = item;
        }
        itemCount++;
        return true;
    }

    public double computeSubtotal() {
        double subtotal = 0;
        if (item1 != null) subtotal += item1.computePrice();
        if (item2 != null) subtotal += item2.computePrice();
        if (item3 != null) subtotal += item3.computePrice();
        return subtotal;
    }
```

```
    public double computeTotal() {
        double subtotal = computeSubtotal();
        if (customer.isStudent()) {
            return subtotal - (subtotal * 0.10);
        }
        return subtotal;
    }

    public double computeTotal(double voucher) {
        double safeVoucher = voucher < 0 ? 0 : voucher;
        double total = computeTotal() - safeVoucher;
        return Math.max(total, 0);
    }

    public void printReceipt() {
        System.out.println("\nOrder #" + orderId + " — "
                + customer.getName()
                + (customer.isStudent() ? " (Student)" : ""));

        if (item1 != null) {
            System.out.printf("  %-40s %.2f%n",
                    item1.getDescription(), item1.computePrice());
        }
        if (item2 != null) {
            System.out.printf("  %-40s %.2f%n",
                    item2.getDescription(), item2.computePrice());
        }
        if (item3 != null) {
            System.out.printf("  %-40s %.2f%n",
                    item3.getDescription(), item3.computePrice());
        }

        double subtotal = computeSubtotal();
        double total = computeTotal();
        double discount = subtotal - total;

        System.out.printf("  Subtotal: %.2f%n", subtotal);
        if (customer.isStudent()) {
            System.out.printf("  Student discount (10%%): -%.2f%n",
discount);
        }
        System.out.printf("  TOTAL: %.2f%n", total);
        customer.addPoints(total);
```

```
        System.out.println("  Loyalty points: " +
customer.getLoyaltyPoints());
    }

    public void printReceipt(double voucher) {
        System.out.println("\nOrder #" + orderId + " — "
                + customer.getName()
                + (customer.isStudent() ? " (Student)" : ""));

        if (item1 != null) {
            System.out.printf("  %-40s %.2f%n",
                    item1.getDescription(), item1.computePrice());
        }
        if (item2 != null) {
            System.out.printf("  %-40s %.2f%n",
                    item2.getDescription(), item2.computePrice());
        }
        if (item3 != null) {
            System.out.printf("  %-40s %.2f%n",
                    item3.getDescription(), item3.computePrice());
        }

        double subtotal = computeSubtotal();
        double discountedTotal = computeTotal();
        double discount = subtotal - discountedTotal;
        double appliedVoucher = Math.min(Math.max(voucher, 0),
discountedTotal);
        double total = discountedTotal - appliedVoucher;

        System.out.printf("  Subtotal: %.2f%n", subtotal);
        if (customer.isStudent()) {
            System.out.printf("  Student discount (10%%): -%.2f%n",
discount);
        }
        System.out.printf("  Voucher: -%.2f%n", appliedVoucher);
        System.out.printf("  TOTAL: %.2f%n", total);
        customer.addPoints(total);
        System.out.println("  Loyalty points: " +
customer.getLoyaltyPoints());
    }
}
```

CafeApp.java

```
public class CafeApp {
    public static void main(String[] args) {
        //Blueprint

        Customer customer1 = new Customer("Ana Reyes", true);
        Customer customer2 = new Customer("Ben Cruz", false);

        Beverage latte = new Beverage("Iced Latte", 95, "B-01", "L");

        Beverage hotChocolate = new Beverage();
        hotChocolate.setName("Hot Chocolate");
        hotChocolate.setItemCode("B-02");
        hotChocolate.setBasePrice(-5);
        hotChocolate.setBasePrice(80);
        hotChocolate.setSize("M");

        Pastry wringkles= new Pastry("wringkles", 45, "P-01", true);

        Pastry cookie = new Pastry();
        cookie.setName("Cookie");
        cookie.setBasePrice(30);
        cookie.setItemCode("P-02");
        cookie.setWarmed(false);

        Meal chickenAdobo = new Meal("Chicken Adobo", 110, "M-01",
true, 1);

        Meal beefTapa = new Meal();
        beefTapa.setName("Beef Tapa");
        beefTapa.setBasePrice(130);
        beefTapa.setItemCode("M-02");
        beefTapa.setUpsized(false);
        beefTapa.setExtraSides(0);
        beefTapa.setExtraSides(-1);

        Order order1 = new Order("1001", customer1);
        order1.addItem(latte);
        order1.addItem(wringkles);
        order1.addItem(chickenAdobo);
        order1.addItem(cookie);  // 4th item → rejected silently
```

```
        order1.printReceipt();

        Order order2 = new Order("1002", customer2);
        order2.addItem(hotChocolate);
        order2.addItem(beefTapa);
        order2.printReceipt(20.0);  // Error 6 FIXED: explicit double
+ method exists

        MenuItem sampleItem = new Beverage("Sample Coffee", 50,
"B-03", "M");
        System.out.println("\nPolymorphism Demonstration:");
        System.out.println(sampleItem.getDescription());
        System.out.printf("Price: %.2f%n",
sampleItem.computePrice());
    }
}
```